universität wien

MULTIOMICS DATA SCIENCE - COLLECTIONS

WHAT WE KNOW SO FAR

- We learned basic data types and how to operate with them
- We stored information in variables and used / manipulated them
- We could manipulate the execution order of commands with control structures
- If / then / else are conditional control structures
- While / for are loop operatores

ANNOUNCEMENT

- For any online exercise submission via mail:
- Please send your submission of a current exercise sheet in **one** Python **.py** file!
- No pdfs, not in two, three, ... files, no jupyter notebooks, please

COLLECTIONS - EXAMPLE

- Assume, we want to store not one scalar information within one variable but a set of dependent information
- Example: We measure over a range of 10 day dayly 3 times the pH value of a substance
- We obtain 30 values in total, which we can treat as elemental information (floats in this case)
- What is the best practice to store and access these numbers?

COLLECTIONS - EXAMPLE

- Intuitively, one could store each value in one variable:

```
ph_value_1 = 7.60
ph_value_2 = 6.81
ph_value_3 = 7.38
...
```

- All follow-up operationen on these values are very restricted and not dynamically

- For example: to compute the average, one needs to statically write the code: ph_avg = ph_value_1 + ph_value_2 + …

- A repeated usage of the code on different data is only partially possible

COLLECTIONS - EXAMPLE

- Intuitively, one could store each value in one variable:

```
ph_value_1 = 7.60
ph_value_2 = 6.81
```

Therefore, never store a set of information like this!

- For example: to compute the average, one needs to statically write the code: ph_avg = ph_value_1 + ph_value_2 + …

- A repeated usage of the code on different data is only partially possible

COLLECTIONS

- To solve this problem, collections were introduced
- Collections can store several information at once
- Builtin collections in Python are:
  - Lists
  - Sets (mathematical)
  - Dictionaries
  - Tuples

LISTS

- Lists are non-elementary objects in Python, which assign several values to a variable in an ordered way

- We already know lists as strings, which contain several information (here characters) in a ordered manner

- Ordered means in this context, that each value in the collection has a numerical index, i.e., 1st value = …, 2nd value = ...

INITIALIZATION OF LISTS

- Lists can be initialized in many ways:

```
my_list1 = list()      # empty list
my_list2 = []          # empty list
```

- Both commands initialize an empty list, they are semantically equal

- We can use the `print` command to output the entire list at any time in a nicely formatted way in the console

- It is also possible to initialize lists with predefined values

```
my_list3 = [4, 10, 3]
```

- Single values are always seperated with a comma , in all collections

universität wien

LISTS

- Im contrast to other programming languages, it is possible in Python to store elements of different data types in the same collection

```
my_list4 = [100.1, "Hello", [10, 2], None]
```

- In this example, we stored a number, a string, another list and a `None` value in a list

LIST ACCESS

- Elements in a list can be accessed in the as with strings by using the [ ] operator

```
elem = my_list[2]      # index starts with 0
```

- All access and slicing operators are analogous to strings

```
l = my_list[-1]        # last element
i = my_list[1:3]       # new list with 1., 2. element
s = my_list[0:3]       # new list with 0. - 2. element
t = my_list[::-1]      # new inverted list
```

INSERTING ELEMENTS IN LISTS

- Single elements can be appended to the end of a list:

```
my_list.append("foo") # string ‘foo‘ will be appended
```

- Inserting a single elementen at a given position:

```
my_list.insert(0, "a")    # inserting string ‘a‘ at position 0
```

- Appending a list to another list:

```
my_list = [0]
my_list.extend([1, 2, 3])     # or
my_list += [1, 2, 3]          # has the same effect
print(my_list)                # output: [0, 1, 2, 3, 1, 2, 3]
```

MANIPULATION OF ELEMENTS IN LISTS

- Slicing also works „left-hand side“, i.e., wenn slicing is executed on the left side of the equation sign:

```
my_list[2] = 10
```

- Of course, the list has to be bigger than the index that you want assign a value to, 2 in this example.

- The size of any collection (or length of a strings) can be obtained with the `len()` function:

```
l = len(my_list)
```

LISTS IN LISTS

- You could almost call it a MATRIX©
- Elements in nested lists can be accessed by using multiple [ ] operators:

```
quasi_matrix = [[0, 1], [2, 3], [4, 5]]
print(quasi_matrix[0][0])     # output: 0
print(quasi_matrix[2][1])     # output: 5
print(quasi_matrix[1])        # output: [2, 3]
```

- Of course, it is possible to construct 3D matrixes, 4D, ...

MANIPULATION OF LISTS

- Slicing assignments really work in all variants:

```
lst = [0, 1, 2, 3, 4]
lst[3:4] = [10, 11]        # output: [0, 1, 2, 10, 11, 4]

lst = [0, 1, 2, 3, 4]
lst[:2] = [5, 6, 7]        # output: [5, 6, 7, 2, 3, 4]

lst = [0, 1, 2, 3, 4]
lst[2:2] = [5, 6, 7]       # output: [0, 1, 5, 6, 7, 2, 3, 4]

lst = [0, 1, 2, 3, 4]
lst[2::-1] = [5, 6, 7]     # output: [7, 6, 5, 3, 4]
```

COPIES OF NON-ELEMENTAL ELEMENTS

- Example:

```
list1 = [0, 1, 2, 3, 4]
list2 = list1
list2[1] = "a"
print(list1)
```

COPIES OF NON-ELEMENTAL ELEMENTS

- Example:

```
list1 = [0, 1, 2, 3, 4]
list2 = list1
list2[1] = "a"
print(list1)
```

- The command `list2 = list1` does not create a new list in memory but only a reference on `list1`.

- A true „deep copy“ you can achieve with the following command:

```
list2 = list(list1)
```

COPIES OF NON-ELEMENTAL ELEMENTS

- Example:

```
list1 = [0, 1, 2, 3, 4]
```

Caution:
When list contain again non-elemental elements, a „deep copy“ also copies references of these elements

- A true „deep copy“ you can achieve with the following command:

```
list2 = list(list1)
```

MORE HELPFUL FUNCTIONS ON LISTS

- Counting the occurrences of an object in a list:

```
c = list1.count(obj)
```

- Returns index of the first occurring object `obj` in der List:

```
i = list1.index(obj)
```

- Sorts the list:

```
list1.sort()        # inplace
```

- Removes and returns the last element in a list:

```
last = list1.pop()
```

universität wien

CREATING PREFILLED LIST

- It is easy to pre-initiate a list with ascending sorted numbers:

```
list1 = list(range(7))
# output: [0, 1, 2, 3, 4, 5, 6]
```

- Or filling a *n*-elemental list with the same values:

```
n = 7
list2 = [0] * n        # [0, 0, 0, 0, 0, 0, 0]
list2 = [0, 2] * 3     # [0, 2, 0, 2, 0, 2]
```

CREATING PREFILLED LIST

- It is easy to pre-initiate a list with ascending sorted numbers:

```
list1 = list(range(7))
# output: [0, 1, 2, 3, 4, 5, 6]
```

- Or filling a *n*-elemental list with the same values:

```
n = 7
list2 = [0] * n        # [0, 0, 0, 0, 0, 0, 0]
list2 = [0, 2] * 3     # [0, 2, 0, 2, 0, 2]
# also works with strings
tv_series = "na" * 16 + "NAMTAB "[::-1]
```

CREATING PREFILLED LIST

- It is easy to pre-initiate a list with ascending sorted numbers:

```
list1 = list(range(7))
# output: [0, 1, 2, 3, 4, 5, 6]
```

- Or filling a *n*-elemental list with the same values:

```
n = 7
list2 = [0] * n        # [0, 0, 0, 0, 0, 0, 0]
list2 = [0, 2] * 3     # [0, 2, 0, 2, 0, 2]
# also works with strings
tv_series = "na" * 16 + "NAMTAB "[::-1]
# "nanananananananananananananananana BATMAN"
```

SAMPLE SOLUTION

- Sample solution of the previous exercise: summing the first *n* squared numbers

```
for n in [5, 42, 123]:
    s = 0
    for i in range(n + 1):
        s += i**2
    print("Sum of squared integers:", s)
```

- Inner code will be executed with three different values for *n*
- Code leaner more efficient
- Maintenance of code easier

SET OBJECTS

- Sets are as lists objects in Python, which can store more than one object
- Sets are designed, to efficiently execute mathematical operations on sets
- In contrast to lists, sets are not ordered, therefore you cannot access their elements with the [ ] operatoren or slice sets
- Sets store equal elements **only once**!

INITIALIZATION AND FILLING OF SETS

- Sets can either be empty initialized of prefilled with a list:

```
set1 = set()        # empty set
set2 = set([0, 1, 2, 3, 4, 4, 4, 4])
print(set2)         # Output: {0, 1, 2, 3, 4}
```

- With these commands, once can add and remove elements from a set:

```
set1.add("a")
set2.remove(2)
```

INITIALIZATION AND FILLING OF SETS

- Sets can either be empty initialized of prefilled with a list:

```
set1 = set()        # empty set
set2 = set([0, 1, 2, 3, 4, 4, 4, 4])
print(set2)         # Output: {0, 1, 2, 3, 4}
```

- With these commands, once can add and remove elements from a set:

```
set1.add("a")
set2.remove(2)
```

- Unbelievable, these function calls are almost self-explanatory ;-)

SET OPERATORS

- The most important operators on sets are the union, intersection, and difference (rest), here an example:

```
set1 = set([0, 1, 2, 3, 4])
set2 = set([3, 4, 5, 6, 7])

union = set1 | set2
print(union)
inter = set1 & set2
print(inter)
rest = set1 - set2          # set1 without set2
print(rest)
```

SET OPERATORS

- The most important operators on sets are the union, intersection, and difference (rest), here an example:

```
set1 = set([0, 1, 2, 3, 4])
set2 = set([3, 4, 5, 6, 7])

union = set1 | set2
print(union)                # Output: {0, 1, 2, 3, 4, 5, 6, 7}
inter = set1 & set2
print(inter)
rest = set1 - set2          # set1 without set2
print(rest)
```

SET OPERATORS

- The most important operators on sets are the union, intersection, and difference (rest), here an example:

```
set1 = set([0, 1, 2, 3, 4])
set2 = set([3, 4, 5, 6, 7])

union = set1 | set2
print(union)                # Output: {0, 1, 2, 3, 4, 5, 6, 7}
inter = set1 & set2
print(inter)                # Output: {3, 4}
rest = set1 - set2          # set1 without set2
print(rest)
```

SET OPERATORS

- The most important operators on sets are the union, intersection, and difference (rest), here an example:

```
set1 = set([0, 1, 2, 3, 4])
set2 = set([3, 4, 5, 6, 7])

union = set1 | set2
print(union)                # Output: {0, 1, 2, 3, 4, 5, 6, 7}
inter = set1 & set2
print(inter)                # Output: {3, 4}
rest = set1 - set2          # set1 without set2
print(rest)                 # Output: {0, 1, 2}
```

DICTIONARIES

- Python is equipped with the dictionaries, which is one of the most useful data structures
- Dictionaries are associative lists, their values are not indexed by a regular number but by any hashable object
- Dictionaries are clever key → value pair data structures
- Since dictionaries are not ordered data structure, slicing is also not possible

INITIALIZATION AND FILLING OF DICTIONARIES

- Here is an example to initialize and fill dictionaries:

```
dict1 = dict()          # empty dictionary
dict2 = {}              # empty dictionary

# Non-empty dictionary
# key and value are separated by a colon :
# Pairs are separated by a comma ,
# Key_1: Value_1, Key_2: Value_2
dict3 = {"a": 22, "up": "down", 21: 42}
```

- **Caution**: due to their limited precision, float numbers are not suitable as keys

MANIPULATION OF DICTIONARIES

- Accessing dictionary elements is as easy as with lists:

```
dict1 = {"foo": "bar", 2: 20}
# access via [] operator
a_value = dict1["foo"]
one_more_value = dict1[2]
```

- Adding and deleting of key / value pairs:

```
dict1["bor"] = "ing"          # adding
del dict1["bor"]              # deleting
```

DICTIONARY AND SET SEARCH

- To check if an element is in a set or dictionary as a key, one can easily use the `in` operator:

```
set1 = {"foo", "bar", 2, 20}
if 2 in set1:
    print("Yes, key ‘2‘ is in the set")

md1 = {1: 2, "d": 14.5}
print(1 in md1)            # Output: True
print(2 in md1)            # Output: False
```

TUPLE

- Tuples are basically the same data structure as lists with one difference: they are immutable
- Once initialized, tuples cannot be manipulated anymore, they need to be re-initialized if you want to manipulate them
- Advantage: tuples can be added in sets or used as keys in dictionaries!

```
t1 = tuple([1, 2, 3]) # declaring a tuple
t2 = (1, 2, 3)        # same effekt
```

- Tuples are ordered as lists and strings

ITERATE OVER COLLECTIONS

- All presented collections are so-called *iterable*
- There are ordered and unordered collections, elements in ordered collections can be accessed via an integer index
- We need iterable collections in a `for` loop (previous week)

ITERATE OVER COLLECTIONS

- One can iterate over lists and tuples in two ways:

```
list1 = [1, 2, 3]
tuple1 = (1, 2, 3)

for i in range(len(tuple1)):
    print(tuple1[i])

for elem in list1:
    print(elem)
```

- In the first approach, we increment an integer index, to access the elements of the collection, in the second approach, the elements are directly assigned to the variable elem

ITERATE OVER COLLECTIONS

- We can also iterate over sets and dictionaries:

```
set1, the_sum = {1, 2, 3}, 0
dict1 = {1: 10, 2: 20, 3: 30}

for el in set1:
    the_sum += el

for e in dict1:
    print(e)
```

```
Output:
```

universität wien

ITERATE OVER COLLECTIONS

- We can also iterate over sets and dictionaries:

```
set1, the_sum = {1, 2, 3}, 0
dict1 = {1: 10, 2: 20, 3: 30}

for el in set1:
    the_sum += el

for e in dict1:
    print(e)
```

```
Output:
1
2
3
```

- Caution: when iterating over a dictionary, you are iterating only over the keys, not the values!

ITERATE OVER DICTIONARIES

- To iterate over the complete pairs of a dictionary, there are two possibilities:

```
dict1 = {1: 2, 3: 4, 5: 6}
for key in dict1:
    value = dict1[key]
    print(key, value)

for key, value in dict1.items():
    print(key, value)
```

- Semantically, both methods are identical

ITERATE WITH AN INCREMENTAL VARIABLE

- Sometimes it is necessary to count the rank of an element, especially in unordered collections. Here, we have two possibilities:

```
i, set1 = 0, {"a", "b", "c", "d"}  # i ist der Iterator
for elem in set1:
    print(i, elem)
    i += 1

for ii, elem in enumerate(set1):
    print(ii, elem)
```

- The function enumerate accepts all possible iterables as input and returns with each iteration an according (counter, value) pair

COMPREHENSIONS FROM FUNCTIONAL PROGRAMMING

- Python offers some concepts from functional programming for all mentioned collections
- Comprehensions are a simplyfied form of describing a collection, which is filled during its initiation
- The only requirement is to have an already present collection as base for a new collection

LIST COMPREHENSIONS

- Example: we already have a list of numbers. We want to multipld (by 2) all values in this list and store them in a new list. Regular approach:

```
new_list, list1 = [], [30, 19, 29, 21, 38, 1, 39, 12, 2]
for x in list1:
    new_list.append(x * 2)
```

- Same example with a *list comprehension*:

```
list1 = [30, 19, 29, 21, 38, 1, 39, 12, 2]
new_list = [x * 2 for x in list1]
```

LIST COMPREHENSIONS

- Next example: we already have a list with numbers. We want to filter this list for numbers, that are bigger than a certain value, square them immediately and store them in a new list. Regular approach:

```
new_list, list1 = [], [30, 19, 29, 21, 38, 1, 39, 12, 2]
for x in list1:
    if x > 20:
        new_list.append(x**2)
```

- Same example with a *list comprehension*:

```
list1 = [30, 19, 29, 21, 38, 1, 39, 12, 2]
new_list = [x**2 for x in list1 if x > 20]
```

TUPLE & SET COMPREHENSIONS

- We use the same example to create a Tuple with a comprehension:

```
list1 = [30, 19, 29, 21, 38, 1, 39, 12, 2]
new_tuple = (x**2 for x in list1 if x > 20)
```

- Caution: the collection symbols () are here different!

- Same example with a set *comprehension*:

```
list1 = [30, 19, 29, 21, 21, 38, 1, 39, 12, 2]
new_set = {x**2 for x in list1 if x > 20}
```

- Although the number 21 is two times present in the initial list, its square number only occurs once in the set

DICTIONARY COMPREHENSIONS

- Since dictionaries store information pairs, their comprehension syntax differs slightly, here an examplel:

```
genes = ["Atg5", "Pparg", "Shh"]
gene_count = {gene: 0 for gene in genes}
# Pre initialization of a dictionary
# to count genen
```

- The comprehension symbols {} are the same is in a set

- The difference is, that at the beginning a key : value pair is defined

DICTIONARY COMPREHENSIONS

- Two lists of same size which contain keys and values separately, can easily be combined to a dictionary:

```
genes = ["Atg5", "Pparg", "Shh"]
values = [2495, 5932, 592]
gene_to_val = {gene: val for gene, val in zip(genes, values)}
```

- The `zip` function accepts as input any number of collections, iterates simultaneously over both, and returns their current elements in a tuple

DICTIONARY COMPREHENSIONS

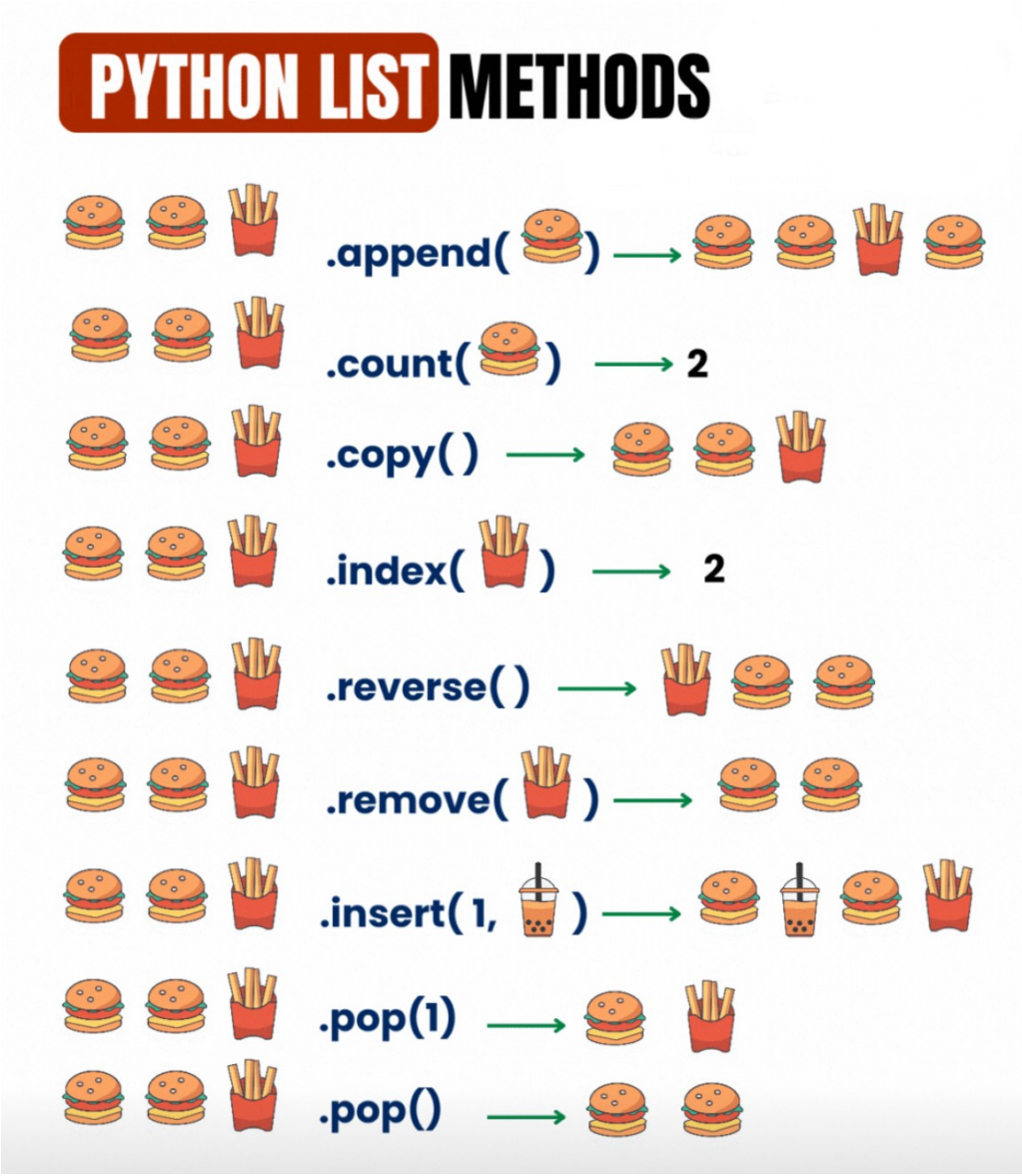
- It is important, that dictionary comprehension contain information pairs

- Another example is the following: we have a list of gen names and we want to store in a dictionary, at which position in the the list the gen name is stored:

```
genes = ["Atg5", "Pparg", "Shh"]
gene_position = {gene: i for i, gene in enumerate(genes)}

print(gene_position) # {"Pparg": 1, "Atg5": 0, "Shh": 2}
```

- Since dictionaries are unordered, they do not necessarily store their pairs in the order in which they were added



Source: *https://includedata.rs/python-lists/*

universität wien